Homework: Diffusion Models

Complete Solutions (Exam Answer Format)

Contents

1 Q1. DDPM: Working Principle, Training and Simplified Objective **3**

1.1 Part 1: Working principle . . . 3

1.2 Part 2: Training procedure (Algorithm 1 of DDPM) . . . 4

1.3 Part 3: Derivation of the simplified training objective . . . 4

1.4 Part 4: Numerical . . . 5

2 Q2. Forward Diffusion Process and the Diffusion Kernel **7**

2.1 Part 1: Forward diffusion process . . . 7

2.2 Part 2: Derivation of the diffusion kernel $q(x_t \mid x_0)$ . . . 7

2.3 Part 3: Role of the noise schedule and timestep selection . . . 7

2.4 Part 4: Numerical . . . 8

3 Q3. Reverse Denoising Process **10**

3.1 Part 1: Reverse denoising process . . . 10

3.2 Part 2: How the neural network estimates the noise . . . 10

3.3 Part 3: Why the true reverse distribution is intractable, and the approximation . . . 10

3.4 Part 4: Numerical . . . 11

4 Q4. U-Net Architecture for Noise Prediction **12**

4.1 Part 1: U-Net architecture . . . 12

4.2 Part 2: Numerical . . . 13

5 Q5. Denoising Score Matching (DSM) **15**

5.1 Part 1: Concept and working principle . . . 15

5.2 Part 2: Numerical . . . 15

6 Q6. Denoising Diffusion Implicit Models (DDIM) **17**

6.1 Part 1: DDIM and the non-Markovian formulation . . . 17

6.2 Part 2: Numerical . . . 18

7 Q7. Classifier Guidance and Classifier-Free Guidance **19**

7.1 Part 1: Conditional generation via the score . . . 19

7.2 Part 2: Classifier guidance (Dhariwal & Nichol, 2021) . . . 19

7.3 Part 3: Classifier-free guidance (Ho & Salimans, 2022) . . . 19

7.4 Part 4: Numerical . . . 20

8 Q8. Latent Diffusion Models (LDM) 22

8.1 Part 1: Architecture and working principle 22

8.2 Part 2: Numerical 23

9 Q9. Accelerating Diffusion Model Sampling 25

9.1 The computational challenge 25

9.2 1. Denoising Diffusion Implicit Models (DDIM) 25

9.3 2. Progressive distillation (Salimans & Ho, 2022) 25

9.4 3. Latent-space diffusion (Rombach et al., 2022) 26

9.5 Comparison 26

10 Q10. Conditional Diffusion Models and Text Conditioning 27

10.1 Part 1: Conditional diffusion models 27

10.2 Part 2: Incorporating conditioning into the U-Net 27

10.3 Part 3: Application – Text-to-image generation (e.g. Stable Diffusion) 27

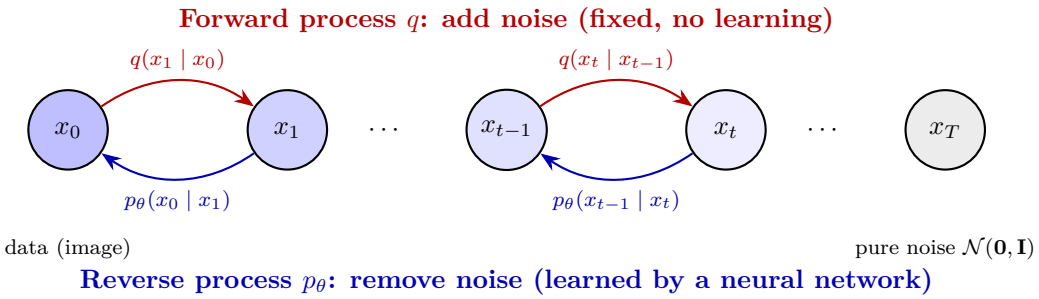
Notation used throughout: β_t = noise variance added at step t; $\alpha_t = 1 - \beta_t$; $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$ (cumulative signal coefficient); $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ = standard Gaussian noise; $\boldsymbol{\epsilon}_\theta(x_t, t)$ = noise predicted by the network; T = total number of diffusion steps.

Q1. DDPM: Working Principle, Training and Simplified Objective

Explain the working principle and training procedure of DDPM and derive the simplified training objective. Given $x_0 = [0.5, -0.2, 0.8]$, $\bar{\alpha}_t = 0.64$, $\boldsymbol{\epsilon} = [0.2, -0.4, 0.6]$, $\boldsymbol{\epsilon}_\theta = [0.1, -0.1, 0.3]$. (a) Noisy vector x_t. (b) Noise prediction loss (squared L2). (c) Estimate $\hat{x}_0$. (d) With $\boldsymbol{\epsilon}'_\theta = [0.18, -0.35, 0.55]$: new loss and % reduction. (e) Reconstruction with $\boldsymbol{\epsilon}'_\theta$; compare reconstruction errors.

Part 1: Working principle

A Denoising Diffusion Probabilistic Model (Ho et al., 2020) is a generative model with two processes:



1. **Forward (diffusion) process q:** A fixed Markov chain that adds a small amount of Gaussian noise at each of T steps (typically $T = 1000$):

$$q(x_t \mid x_{t-1}) = \mathcal{N}(x_t;\ \sqrt{1-\beta_t}\, x_{t-1},\ \beta_t \mathbf{I}), \qquad q(x_{1:T} \mid x_0) = \prod_{t=1}^{T} q(x_t \mid x_{t-1}).$$

   The variances $\beta_1 < \beta_2 < \cdots < \beta_T$ (e.g. linearly from 10^{-4} to 0.02) are chosen so that x_T is almost pure Gaussian noise. A key property (derived in Q2) is the closed-form *diffusion kernel*:

$$q(x_t \mid x_0) = \mathcal{N}(x_t;\ \sqrt{\bar{\alpha}_t}\, x_0,\ (1-\bar{\alpha}_t)\mathbf{I}) \quad \Longleftrightarrow \quad x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \boldsymbol{\epsilon}.$$

2. **Reverse (denoising) process p_θ:** A learned Markov chain that starts from $x_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and removes noise step by step:

$$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}(x_{t-1};\ \mu_\theta(x_t, t),\ \sigma_t^2 \mathbf{I}), \qquad p_\theta(x_{0:T}) = p(x_T) \prod_{t=1}^{T} p_\theta(x_{t-1} \mid x_t).$$

   A neural network (usually a U-Net) is trained to predict the noise $\boldsymbol{\epsilon}_\theta(x_t, t)$ contained in x_t; from it the mean μ_θ is computed.

3. **Generation:** sample $x_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and apply the reverse step T times to obtain a new data sample x_0.

Part 2: Training procedure (Algorithm 1 of DDPM)

Training

repeat

1. Take a clean training sample $x_0 \sim q(x_0)$ from the dataset.
2. Sample a timestep uniformly: $t \sim \text{Uniform}\{1, \dots, T\}$.
3. Sample noise $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.
4. Create the noisy sample in one step: $x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \boldsymbol{\epsilon}$.
5. Predict the noise: $\boldsymbol{\epsilon}_\theta(x_t, t)$.
6. Take a gradient descent step on $\nabla_\theta \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(x_t, t) \right\|^2$.

until converged

Sampling

$x_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$; for $t = T, \dots, 1$: $z \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $z = \mathbf{0}$; $x_{t-1} = \frac{1}{\sqrt{\alpha_t}}\Big(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(x_t, t)\Big) + \sigma_t z$; return x_0.

Part 3: Derivation of the simplified training objective

Step 1 – Variational lower bound (ELBO). We want to maximize $\log p_\theta(x_0)$. Using the forward process as the variational distribution and Jensen's inequality:

$$\log p_\theta(x_0) = \log \int p_\theta(x_{0:T})\, dx_{1:T} = \log \mathbb{E}_{q(x_{1:T}|x_0)}\left[\frac{p_\theta(x_{0:T})}{q(x_{1:T} \mid x_0)}\right] \;\geq\; \mathbb{E}_q\left[\log \frac{p_\theta(x_{0:T})}{q(x_{1:T} \mid x_0)}\right].$$

So we minimize $L = \mathbb{E}_q\left[-\log \frac{p_\theta(x_{0:T})}{q(x_{1:T} \mid x_0)}\right] \geq -\log p_\theta(x_0)$.

Step 2 – Decompose into KL terms. Using Bayes' rule $q(x_t \mid x_{t-1}) = q(x_t \mid x_{t-1}, x_0) = \frac{q(x_{t-1} \mid x_t, x_0)\, q(x_t \mid x_0)}{q(x_{t-1} \mid x_0)}$ and cancelling the telescoping terms:

$$L = \underbrace{D_{\text{KL}}(q(x_T \mid x_0) \,\|\, p(x_T))}_{L_T} + \sum_{t=2}^{T} \underbrace{D_{\text{KL}}(q(x_{t-1} \mid x_t, x_0) \,\|\, p_\theta(x_{t-1} \mid x_t))}_{L_{t-1}} \underbrace{- \log p_\theta(x_0 \mid x_1)}_{L_0}.$$

L_T has no trainable parameters (both distributions are fixed and $\approx \mathcal{N}(\mathbf{0}, \mathbf{I})$), so it is a constant.

Step 3 – The forward posterior is Gaussian. Conditioning on x_0 makes the posterior tractable:

$$q(x_{t-1} \mid x_t, x_0) = \mathcal{N}\big(x_{t-1};\ \tilde{\mu}_t(x_t, x_0),\ \tilde{\beta}_t \mathbf{I}\big),$$
$$\tilde{\mu}_t = \frac{\sqrt{\bar{\alpha}_{t-1}}\,\beta_t}{1-\bar{\alpha}_t} x_0 + \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t} x_t, \qquad \tilde{\beta}_t = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\beta_t.$$

Step 4 – Express $\tilde{\mu}_t$ using the noise. From the kernel, $x_0 = \frac{1}{\sqrt{\bar{\alpha}_t}}(x_t - \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon})$. Substitute into $\tilde{\mu}_t$:

- Coefficient of x_t (using $\frac{\sqrt{\bar{\alpha}_{t-1}}}{\sqrt{\bar{\alpha}_t}} = \frac{1}{\sqrt{\alpha_t}}$ and $\beta_t = 1 - \alpha_t$):

$$\frac{\sqrt{\bar{\alpha}_{t-1}}\,\beta_t}{(1-\bar{\alpha}_t)\sqrt{\bar{\alpha}_t}} + \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t} = \frac{\beta_t + \alpha_t(1-\bar{\alpha}_{t-1})}{\sqrt{\alpha_t}(1-\bar{\alpha}_t)} = \frac{1-\alpha_t+\alpha_t-\bar{\alpha}_t}{\sqrt{\alpha_t}(1-\bar{\alpha}_t)} = \frac{1}{\sqrt{\alpha_t}}.$$

- Coefficient of $\boldsymbol{\epsilon}$: $-\dfrac{\sqrt{\bar{\alpha}_{t-1}}\,\beta_t\sqrt{1-\bar{\alpha}_t}}{(1-\bar{\alpha}_t)\sqrt{\alpha_t}} = -\dfrac{\beta_t}{\sqrt{\alpha_t}\sqrt{1-\bar{\alpha}_t}}$.

$$\Rightarrow \quad \tilde{\mu}_t = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\,\boldsymbol{\epsilon}\right).$$

Step 5 – Parameterize the model mean in the same form (the network predicts the noise):

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\,\boldsymbol{\epsilon}_\theta(x_t, t)\right).$$

Step 6 – KL between two Gaussians with the same fixed variance σ_t^2:

$$L_{t-1} = \frac{1}{2\sigma_t^2}\|\tilde{\mu}_t - \mu_\theta\|^2 + C = \frac{1}{2\sigma_t^2}\cdot\frac{\beta_t^2}{\alpha_t(1-\bar{\alpha}_t)}\,\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\,x_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon},\ t)\|^2 + C.$$

Step 7 – Simplify. Ho et al. found that dropping the time-dependent weight $\dfrac{\beta_t^2}{2\sigma_t^2\alpha_t(1-\bar{\alpha}_t)}$ gives better sample quality (it down-weights the easy small-t terms). This gives the **simplified objective**:

$$\boxed{L_{\text{simple}}(\theta) = \mathbb{E}_{t\sim U\{1,T\},\ x_0\sim q(x_0),\ \boldsymbol{\epsilon}\sim\mathcal{N}(\mathbf{0},\mathbf{I})}\left[\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\,x_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon},\ t)\|^2\right]}$$

i.e. a simple mean-squared error between the true noise and the predicted noise.

Part 4: Numerical

Given: $x_0 = [0.5, -0.2, 0.8]$, $\bar{\alpha}_t = 0.64$, $\boldsymbol{\epsilon} = [0.2, -0.4, 0.6]$, $\boldsymbol{\epsilon}_\theta = [0.1, -0.1, 0.3]$.
Coefficients: $\sqrt{\bar{\alpha}_t} = \sqrt{0.64} = 0.8, \quad \sqrt{1-\bar{\alpha}_t} = \sqrt{0.36} = 0.6$.

(a) Noisy data vector x_t

$$\begin{aligned} x_t &= \sqrt{\bar{\alpha}_t}\,x_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon} = 0.8\,x_0 + 0.6\,\boldsymbol{\epsilon} \\ 0.8\,x_0 &= [0.8\times 0.5,\ 0.8\times(-0.2),\ 0.8\times 0.8] = [0.40,\ -0.16,\ 0.64] \\ 0.6\,\boldsymbol{\epsilon} &= [0.6\times 0.2,\ 0.6\times(-0.4),\ 0.6\times 0.6] = [0.12,\ -0.24,\ 0.36] \\ x_t &= [0.40+0.12,\ -0.16-0.24,\ 0.64+0.36] = [0.52,\ -0.40,\ 1.00] \end{aligned}$$

$$\boxed{x_t = [0.52,\ -0.40,\ 1.00]}$$

(b) Noise prediction loss (squared L2 norm)

$$\begin{aligned} \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta &= [0.2-0.1,\ -0.4-(-0.1),\ 0.6-0.3] = [0.1,\ -0.3,\ 0.3] \\ L = \|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta\|_2^2 &= (0.1)^2 + (-0.3)^2 + (0.3)^2 = 0.01 + 0.09 + 0.09 = 0.19 \end{aligned}$$

$$\boxed{L = 0.19 \quad \text{(mean over the 3 elements: } 0.19/3 = 0.0633\text{)}}$$

(c) Estimate of the original data $\hat{x}_0$

Rearranging the kernel and replacing the true noise by the predicted noise:

$$\begin{aligned} \hat{x}_0 &= \frac{x_t - \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}_\theta}{\sqrt{\bar{\alpha}_t}} = \frac{x_t - 0.6\,\boldsymbol{\epsilon}_\theta}{0.8} \\ 0.6\,\boldsymbol{\epsilon}_\theta &= [0.6\times 0.1,\ 0.6\times(-0.1),\ 0.6\times 0.3] = [0.06,\ -0.06,\ 0.18] \\ x_t - 0.6\,\boldsymbol{\epsilon}_\theta &= [0.52-0.06,\ -0.40+0.06,\ 1.00-0.18] = [0.46,\ -0.34,\ 0.82] \\ \hat{x}_0 &= \left[\frac{0.46}{0.8},\ \frac{-0.34}{0.8},\ \frac{0.82}{0.8}\right] = [0.575,\ -0.425,\ 1.025] \end{aligned}$$

$\hat{x}_0 = [0.575,\ -0.425,\ 1.025]$ (true $x_0 = [0.5, -0.2, 0.8]$)

Reconstruction error: $\hat{x}_0 - x_0 = [0.075,\ -0.225,\ 0.225]$, $\|\hat{x}_0 - x_0\|^2 = 0.005625 + 0.050625 + 0.050625 = 0.106875$.

(d) Updated loss and percentage reduction

$$\boldsymbol{\epsilon} - \boldsymbol{\epsilon}'_\theta = [0.2 - 0.18,\ -0.4 - (-0.35),\ 0.6 - 0.55] = [0.02,\ -0.05,\ 0.05]$$
$$L' = (0.02)^2 + (-0.05)^2 + (0.05)^2 = 0.0004 + 0.0025 + 0.0025 = 0.0054$$
$$\text{Reduction} = \frac{L - L'}{L} \times 100\% = \frac{0.19 - 0.0054}{0.19} \times 100\% = \frac{0.1846}{0.19} \times 100\% = 97.16\%$$

Updated loss $L' = 0.0054$; percentage reduction $\approx 97.16\%$.

(e) Reconstruction with the updated prediction

$$0.6\,\boldsymbol{\epsilon}'_\theta = [0.6 \times 0.18,\ 0.6 \times (-0.35),\ 0.6 \times 0.55] = [0.108,\ -0.210,\ 0.330]$$
$$x_t - 0.6\,\boldsymbol{\epsilon}'_\theta = [0.52 - 0.108,\ -0.40 + 0.210,\ 1.00 - 0.330] = [0.412,\ -0.190,\ 0.670]$$
$$\hat{x}'_0 = \left[\frac{0.412}{0.8},\ \frac{-0.190}{0.8},\ \frac{0.670}{0.8}\right] = [0.515,\ -0.2375,\ 0.8375]$$

Reconstruction error:

$$\hat{x}'_0 - x_0 = [0.515 - 0.5,\ -0.2375 + 0.2,\ 0.8375 - 0.8] = [0.015,\ -0.0375,\ 0.0375]$$
$$\|\hat{x}'_0 - x_0\|^2 = 0.000225 + 0.00140625 + 0.00140625 = 0.0030375$$

	Reconstruction $\hat{x}_0$	Squared error	MSE (per element)
Initial ($\boldsymbol{\epsilon}_\theta$)	$[0.575, -0.425, 1.025]$	0.106875	0.035625
Updated ($\boldsymbol{\epsilon}'_\theta$)	$[0.515, -0.2375, 0.8375]$	0.0030375	0.0010125

$$\text{Reduction in reconstruction error} = \frac{0.106875 - 0.0030375}{0.106875} \times 100\% = 97.16\%$$

$\hat{x}'_0 = [0.515,\ -0.2375,\ 0.8375]$; squared reconstruction error falls from 0.106875 to 0.0030375 (a 97.16% reduction, about 35× smaller). The updated reconstruction is much closer to the true $x_0 = [0.5, -0.2, 0.8]$.

Observation: $\hat{x}_0 - x_0 = -\dfrac{\sqrt{1-\bar{\alpha}_t}}{\sqrt{\bar{\alpha}_t}}(\boldsymbol{\epsilon}_\theta - \boldsymbol{\epsilon})$, so the reconstruction error $= \left(\dfrac{0.6}{0.8}\right)^2 \times$ noise loss $= 0.5625\times$ noise loss. Check: $0.5625 \times 0.19 = 0.106875$ and $0.5625 \times 0.0054 = 0.0030375$ ✓. This shows why minimizing the noise-prediction loss directly improves the recovery of clean data.

Q2. Forward Diffusion Process and the Diffusion Kernel

Explain the forward diffusion process, derive the diffusion kernel relating x_t to x_0, and explain the role of the noise schedule and timestep selection. Pixel $x_0 = 0.8$; $x_t = \sqrt{1-\beta_t}\, x_{t-1} + \sqrt{\beta_t}\, \epsilon_t$; $(\beta_1, \beta_2, \beta_3) = (0.1, 0.2, 0.3)$; $(\epsilon_1, \epsilon_2, \epsilon_3) = (0.2, -0.3, 0.5)$. (a) x_1, x_2, x_3 sequentially. (b) $\bar{\alpha}_t$ at each step. (c) x_3 directly from the kernel with combined noise; compare. (d) Effect of increasing t on signal and noise; why intensity need not decrease monotonically.

Part 1: Forward diffusion process

The forward process gradually destroys the structure of a data sample by adding Gaussian noise over T steps. It is a *fixed* Markov chain (no learnable parameters):

$$q(x_t \mid x_{t-1}) = \mathcal{N}(x_t;\ \sqrt{1-\beta_t}\, x_{t-1},\ \beta_t \mathbf{I}) \quad \Longleftrightarrow \quad x_t = \sqrt{1-\beta_t}\, x_{t-1} + \sqrt{\beta_t}\, \epsilon_t, \quad \epsilon_t \sim \mathcal{N}(0, \mathbf{I}).$$

- At each step the previous sample is **scaled down** by $\sqrt{1-\beta_t} < 1$ and a small amount of **noise** with variance β_t is added.
- The scaling keeps the variance bounded: if $\mathrm{Var}(x_{t-1}) = 1$, then $\mathrm{Var}(x_t) = (1-\beta_t) \cdot 1 + \beta_t = 1$ ("variance-preserving").
- As $t \to T$, the information of x_0 disappears and $x_T \approx \mathcal{N}(\mathbf{0}, \mathbf{I})$, a distribution we can sample from easily.

Part 2: Derivation of the diffusion kernel $q(x_t \mid x_0)$

Let $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$. Then $x_t = \sqrt{\alpha_t}\, x_{t-1} + \sqrt{1-\alpha_t}\, \epsilon_t$.

Step 1 – Unroll one step. Substitute $x_{t-1} = \sqrt{\alpha_{t-1}}\, x_{t-2} + \sqrt{1-\alpha_{t-1}}\, \epsilon_{t-1}$:

$$x_t = \sqrt{\alpha_t \alpha_{t-1}}\, x_{t-2} + \underbrace{\sqrt{\alpha_t(1-\alpha_{t-1})}\, \epsilon_{t-1} + \sqrt{1-\alpha_t}\, \epsilon_t}_{\text{sum of two independent Gaussians}}.$$

Step 2 – Merge the Gaussians. If $a \sim \mathcal{N}(0, \sigma_1^2)$ and $b \sim \mathcal{N}(0, \sigma_2^2)$ are independent, then $a + b \sim \mathcal{N}(0, \sigma_1^2 + \sigma_2^2)$. The variance of the noise part is

$$\alpha_t(1-\alpha_{t-1}) + (1-\alpha_t) = \alpha_t - \alpha_t\alpha_{t-1} + 1 - \alpha_t = 1 - \alpha_t\alpha_{t-1}.$$

Hence $x_t = \sqrt{\alpha_t\alpha_{t-1}}\, x_{t-2} + \sqrt{1-\alpha_t\alpha_{t-1}}\, \bar{\epsilon}$, with $\bar{\epsilon} \sim \mathcal{N}(0, \mathbf{I})$.

Step 3 – Repeat down to x_0 (induction). Assume $x_{t-1} = \sqrt{\bar{\alpha}_{t-1}}\, x_0 + \sqrt{1-\bar{\alpha}_{t-1}}\, \bar{\epsilon}'$. Then

$$x_t = \sqrt{\alpha_t}\sqrt{\bar{\alpha}_{t-1}}\, x_0 + \sqrt{\alpha_t(1-\bar{\alpha}_{t-1})}\, \bar{\epsilon}' + \sqrt{1-\alpha_t}\, \epsilon_t,$$

with noise variance $\alpha_t - \alpha_t\bar{\alpha}_{t-1} + 1 - \alpha_t = 1 - \bar{\alpha}_t$. Therefore

$$\boxed{x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \bar{\epsilon}, \qquad q(x_t \mid x_0) = \mathcal{N}(x_t;\ \sqrt{\bar{\alpha}_t}\, x_0,\ (1-\bar{\alpha}_t)\mathbf{I})}$$

Importance: During training we can jump directly to any timestep t in one step, without simulating the t intermediate steps.

Part 3: Role of the noise schedule and timestep selection

Noise schedule $\{\beta_t\}_{t=1}^{T}$:

- Controls how fast information is destroyed: $\bar{\alpha}_t$ (signal) decreases from ≈ 1 to ≈ 0; signal-to-noise ratio $\mathrm{SNR}(t) = \bar{\alpha}_t/(1-\bar{\alpha}_t)$ decreases monotonically.

- **Small** β_t ensures each reverse step $q(x_{t-1} \mid x_t)$ is close to Gaussian, so the Gaussian reverse model is accurate.
- $\bar{\alpha}_T \approx 0$ is needed so that x_T matches the prior $\mathcal{N}(\mathbf{0}, \mathbf{I})$ used at sampling time.
- **Linear schedule** (DDPM: β_t from 10^{-4} to 0.02, $T = 1000$) destroys information quickly near the end, wasting many steps on almost-pure noise. The **cosine schedule** (Nichol & Dhariwal) $\bar{\alpha}_t = \dfrac{f(t)}{f(0)}$, $f(t) = \cos^2\Big(\dfrac{t/T + s}{1+s} \cdot \dfrac{\pi}{2}\Big)$ decreases more gradually and improves quality, especially for low-resolution images.

Timestep selection during training:

- For each training example, t is sampled *uniformly* from $\{1, \ldots, T\}$ (not sequentially). Thanks to the kernel, x_t is created in one step.
- This trains one network on all noise levels simultaneously, which is efficient and unbiased (each term of the loss is a random sample of the full sum over t).
- The network receives t (as a sinusoidal embedding) so it knows the noise level; small t teaches fine detail removal, large t teaches global structure.
- Importance sampling or loss weighting over t (e.g. by SNR) can further reduce variance and improve results.

Part 4: Numerical

Given: $x_0 = 0.8$; $\beta_1 = 0.1$, $\beta_2 = 0.2$, $\beta_3 = 0.3$; $\epsilon_1 = 0.2$, $\epsilon_2 = -0.3$, $\epsilon_3 = 0.5$.
Therefore $\alpha_1 = 0.9$, $\alpha_2 = 0.8$, $\alpha_3 = 0.7$.

(a) Sequential forward diffusion

Timestep 1: $\sqrt{1-\beta_1} = \sqrt{0.9} = 0.948683$, $\sqrt{\beta_1} = \sqrt{0.1} = 0.316228$.

$$x_1 = 0.948683 \times 0.8 + 0.316228 \times 0.2 = 0.758947 + 0.063246 = 0.822192$$

Timestep 2: $\sqrt{1-\beta_2} = \sqrt{0.8} = 0.894427$, $\sqrt{\beta_2} = \sqrt{0.2} = 0.447214$.

$$x_2 = 0.894427 \times 0.822192 + 0.447214 \times (-0.3) = 0.735391 - 0.134164 = 0.601227$$

Timestep 3: $\sqrt{1-\beta_3} = \sqrt{0.7} = 0.836660$, $\sqrt{\beta_3} = \sqrt{0.3} = 0.547723$.

$$x_3 = 0.836660 \times 0.601227 + 0.547723 \times 0.5 = 0.503023 + 0.273861 = 0.776884$$

$x_1 \approx 0.8222, \quad x_2 \approx 0.6012, \quad x_3 \approx 0.7769$

(b) Cumulative signal coefficient $\bar{\alpha}_t = \prod_{s=1}^{t}(1-\beta_s)$

$$\bar{\alpha}_1 = 0.9$$
$$\bar{\alpha}_2 = 0.9 \times 0.8 = 0.72$$
$$\bar{\alpha}_3 = 0.9 \times 0.8 \times 0.7 = 0.504$$

t	$\bar{\alpha}_t$	$\sqrt{\bar{\alpha}_t}$ (signal coeff.)	$1-\bar{\alpha}_t$ (noise var.)	$\sqrt{1-\bar{\alpha}_t}$ (noise std)	SNR $= \bar{\alpha}_t/(1-\bar{\alpha}_t)$
1	0.900	0.9487	0.100	0.3162	9.000
2	0.720	0.8485	0.280	0.5292	2.571
3	0.504	0.7099	0.496	0.7043	1.016

$\bar{\alpha}_1 = 0.9, \quad \bar{\alpha}_2 = 0.72, \quad \bar{\alpha}_3 = 0.504$

(c) Direct computation of x_3 using the diffusion kernel

Step 1 – Combine the three noise samples. Unrolling the recursion:

$$x_3 = \sqrt{\alpha_3\alpha_2\alpha_1}\, x_0 + \sqrt{\alpha_3\alpha_2\beta_1}\, \epsilon_1 + \sqrt{\alpha_3\beta_2}\, \epsilon_2 + \sqrt{\beta_3}\, \epsilon_3.$$

Noise coefficients:

$$\sqrt{\alpha_3\alpha_2\beta_1} = \sqrt{0.7 \times 0.8 \times 0.1} = \sqrt{0.056} = 0.236643$$
$$\sqrt{\alpha_3\beta_2} = \sqrt{0.7 \times 0.2} = \sqrt{0.14} = 0.374166$$
$$\sqrt{\beta_3} = \sqrt{0.3} = 0.547723$$

Check of variance: $0.056 + 0.14 + 0.3 = 0.496 = 1 - \bar{\alpha}_3$ ✓.

Total noise contribution:

$$0.236643 \times 0.2 + 0.374166 \times (-0.3) + 0.547723 \times 0.5 = 0.047329 - 0.112250 + 0.273861 = 0.208940.$$

Equivalent single (combined) standard Gaussian noise:

$$\bar{\epsilon} = \frac{0.208940}{\sqrt{1 - \bar{\alpha}_3}} = \frac{0.208940}{\sqrt{0.496}} = \frac{0.208940}{0.704273} = 0.296675.$$

Step 2 – Apply the kernel:

$$\begin{aligned} x_3 &= \sqrt{\bar{\alpha}_3}\, x_0 + \sqrt{1 - \bar{\alpha}_3}\, \bar{\epsilon} \\ &= \sqrt{0.504} \times 0.8 + \sqrt{0.496} \times 0.296675 \\ &= 0.709930 \times 0.8 + 0.704273 \times 0.296675 \\ &= 0.567944 + 0.208940 = 0.776884 \end{aligned}$$

Direct kernel: $x_3 = 0.776884$. Sequential (part a): $x_3 = 0.776884$.
The two results are **identical**, which confirms that $q(x_3 \mid x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_3}\, x_0,\ (1 - \bar{\alpha}_3))$ is exactly equivalent to applying the three forward steps one by one, provided the combined noise $\bar{\epsilon}$ is built from the same $\epsilon_1, \epsilon_2, \epsilon_3$.

(d) Effect of increasing the timestep

t	Signal part $\sqrt{\bar{\alpha}_t}\, x_0$	Noise std $\sqrt{1 - \bar{\alpha}_t}$	Observed x_t
0	0.8000	0	0.8000
1	0.7589	0.3162	0.8222
2	0.6788	0.5292	0.6012
3	0.5679	0.7043	0.7769

- **Signal decreases:** The contribution of the original pixel falls steadily: $0.8 \to 0.759 \to 0.679 \to 0.568$, because $\sqrt{\bar{\alpha}_t}$ is a product of numbers less than 1.
- **Noise increases:** The noise standard deviation grows: $0 \to 0.316 \to 0.529 \to 0.704$. The SNR falls from 9 to about 1; at $t = 3$ signal and noise have nearly equal weight. As $t \to T$, $\bar{\alpha}_t \to 0$ and x_t becomes pure noise.
- **Non-monotonic intensity:** The observed value goes $0.800 \to 0.822$ (up) $\to 0.601$ (down) $\to 0.777$ (up). This happens because x_t is the sum of a decreasing deterministic signal and a *random* noise term whose sign varies: $\epsilon_1 = 0.2 > 0$ and $\epsilon_3 = 0.5 > 0$ push the value up, while $\epsilon_2 = -0.3$ pulls it down. Only the *expected value* $\mathbb{E}[x_t] = \sqrt{\bar{\alpha}_t}\, x_0$ decreases monotonically; individual samples fluctuate around it with growing spread.

Q3. Reverse Denoising Process

Describe the reverse denoising process; how a network estimates noise; why the reverse conditional is hard to compute and how it is approximated. Reverse step: $x_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left[x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\boldsymbol{\epsilon}_\theta(x_t, t)\right] + \sigma_t z$. Given $x_t = 0.75$, $\alpha_t = 0.8$, $\beta_t = 0.2$, $\bar{\alpha}_t = 0.5$, $\boldsymbol{\epsilon}_\theta = 0.6$, $\sigma_t = 0.15$, $z = -0.4$. (a) Noise correction term. (b) x_{t-1}. (c) x_{t-1} without the stochastic term. (d) Role of the stochastic component.

Part 1: Reverse denoising process

The reverse process generates data by inverting the forward process: starting from pure noise $x_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ it removes a small amount of noise at each step, $x_T \to x_{T-1} \to \cdots \to x_0$. Each step is a Gaussian:

$$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}\big(x_{t-1};\ \mu_\theta(x_t, t),\ \sigma_t^2 \mathbf{I}\big), \qquad \mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\boldsymbol{\epsilon}_\theta(x_t, t)\right),$$

with $\sigma_t^2 = \beta_t$ or $\sigma_t^2 = \tilde{\beta}_t = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\beta_t$. A sample is drawn as $x_{t-1} = \mu_\theta(x_t, t) + \sigma_t z$, $z \sim \mathcal{N}(0, \mathbf{I})$ (with $z = 0$ at the last step).

Part 2: How the neural network estimates the noise

- **Input:** the noisy sample x_t and the timestep t. The timestep is encoded with a sinusoidal embedding followed by an MLP and injected into every residual block, so the network knows the noise level.
- **Network:** usually a U-Net with residual blocks and self-attention (Q4). Its output has the same shape as x_t: the predicted noise $\boldsymbol{\epsilon}_\theta(x_t, t)$.
- **Training:** it learns to answer "which noise $\boldsymbol{\epsilon}$ was added to produce x_t?" by minimizing $\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon},\ t)\|^2$ over random x_0, t, $\boldsymbol{\epsilon}$.
- **Use:** the predicted noise gives (i) an estimate of the clean image $\hat{x}_0 = (x_t - \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}_\theta)/\sqrt{\bar{\alpha}_t}$ and (ii) the reverse mean μ_θ. Equivalently, $\boldsymbol{\epsilon}_\theta$ estimates the score: $\nabla_{x_t} \log q(x_t) \approx -\boldsymbol{\epsilon}_\theta(x_t, t)/\sqrt{1-\bar{\alpha}_t}$.

Part 3: Why the true reverse distribution is intractable, and the approximation

- By Bayes' rule, $q(x_{t-1} \mid x_t) = \frac{q(x_t \mid x_{t-1})\, q(x_{t-1})}{q(x_t)}$, where $q(x_{t-1}) = \int q(x_{t-1} \mid x_0)\, q(x_0)\, dx_0$ and $q(x_t)$ are *marginals over the entire unknown data distribution* $q(x_0)$. Computing them requires integrating over all possible images, which is impossible in high dimensions.
- In general $q(x_{t-1} \mid x_t)$ is a complicated, multimodal distribution (many clean images could have produced the same x_t).
- **Tractable if conditioned on x_0:** $q(x_{t-1} \mid x_t, x_0) = \mathcal{N}(\tilde{\mu}_t, \tilde{\beta}_t \mathbf{I})$ is a closed-form Gaussian (Q1, Step 3), but x_0 is unknown at generation time.
- **Approximation:**
  1. When β_t is small, the true reverse step is approximately Gaussian (Feller, 1949), so it is modelled as $p_\theta(x_{t-1} \mid x_t) = \mathcal{N}(\mu_\theta, \sigma_t^2 \mathbf{I})$.
  2. The mean is learned by matching the tractable posterior mean $\tilde{\mu}_t$ (minimizing $D_{\mathrm{KL}}(q(x_{t-1} \mid x_t, x_0) \,\|\, p_\theta(x_{t-1} \mid x_t))$ in the ELBO), which reduces to predicting the noise $\boldsymbol{\epsilon}$, i.e. effectively predicting x_0 from x_t.
  3. The variance is fixed ($\sigma_t^2 = \beta_t$ or $\tilde{\beta}_t$) or learned (Improved DDPM).

Part 4: Numerical

Given: $x_t = 0.75$, $\alpha_t = 0.8$, $\beta_t = 0.2$, $\bar{\alpha}_t = 0.5$, $\boldsymbol{\epsilon}_\theta(x_t, t) = 0.6$, $\sigma_t = 0.15$, $z = -0.4$.

(a) Noise correction term

$$\sqrt{1-\bar{\alpha}_t} = \sqrt{1-0.5} = \sqrt{0.5} = 0.707107$$
$$\frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} = \frac{0.2}{0.707107} = 0.282843$$
$$\text{Noise correction} = \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\boldsymbol{\epsilon}_\theta(x_t, t) = 0.282843 \times 0.6 = 0.169706$$

Noise correction term ≈ 0.1697

(b) Reverse diffusion value x_{t-1}

$$x_t - \text{correction} = 0.75 - 0.169706 = 0.580294$$
$$\frac{1}{\sqrt{\alpha_t}} = \frac{1}{\sqrt{0.8}} = \frac{1}{0.894427} = 1.118034$$
$$\mu_\theta = 1.118034 \times 0.580294 = 0.648789$$
$$\sigma_t z = 0.15 \times (-0.4) = -0.06$$
$$x_{t-1} = \mu_\theta + \sigma_t z = 0.648789 - 0.06 = 0.588789$$

$x_{t-1} \approx 0.5888$

(c) x_{t-1} without the stochastic term ($z = 0$)

$$x_{t-1} = \mu_\theta = \frac{1}{\sqrt{0.8}}(0.75 - 0.169706) = 1.118034 \times 0.580294 = 0.648789$$

Deterministic $x_{t-1} \approx 0.6488$; the stochastic term shifted the value by $\sigma_t z = -0.06$ (from 0.6488 to 0.5888).

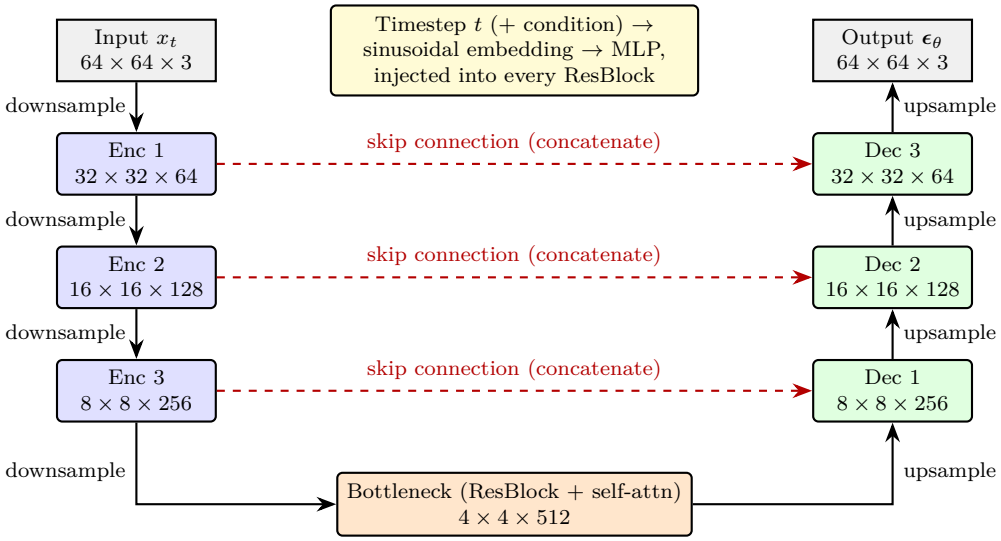
(d) Role of the stochastic component $\sigma_t z$

1. **Correct sampling of a distribution:** The reverse step is a Gaussian $\mathcal{N}(\mu_\theta, \sigma_t^2)$, not a single point. Adding $\sigma_t z$ draws a sample from it; without it, DDPM would not sample from the modelled distribution $p_\theta(x_{t-1} \mid x_t)$.
2. **Diversity:** Different random z at each step lead to different final images from the same starting noise, giving varied outputs instead of collapsing to an "average" image.
3. **Error correction:** Fresh noise lets the sampler recover from small prediction errors made at earlier steps (errors are partially "washed out" instead of accumulating along a fixed path), so stochastic DDPM sampling is robust.
4. **Matches the forward process:** Noise is removed gradually; reinjecting a small, decreasing amount keeps x_{t-1} at the correct noise level $\approx \sqrt{1-\bar{\alpha}_{t-1}}$ expected by the network at the next step.
5. **Final step:** At $t = 1$, $z = 0$ so that the output x_0 is a clean image. Setting $\sigma_t = 0$ at all steps gives deterministic sampling (DDIM, Q6), which can use fewer steps but loses this stochasticity.

Q4. U-Net Architecture for Noise Prediction

Explain the U-Net architecture used in diffusion models and the functions of its components. Feature maps: Input $64\times64\times3$; Enc1 $32\times32\times64$; Enc2 $16\times16\times128$; Enc3 $8\times8\times256$; Bottleneck $4\times4\times512$; Dec1 $8\times8\times256$; Dec2 $16\times16\times128$; Dec3 $32\times32\times64$; Output $64\times64\times3$. (a) Elements in Enc2 and Bottleneck. (b) Dimensions after concatenating $8\times8\times256$ (upsampled) with Enc3. (c) Elements of (b) vs before concatenation. (d) Residual output for input $[0.2, 0.4, 0.6]$, transform $[0.1, -0.2, 0.3]$. (e) MSE for $\boldsymbol{\epsilon} = [0.2, 0.5, -0.3, 0.1]$, $\boldsymbol{\epsilon}_\theta = [0.1, 0.4, -0.1, 0.2]$. (f) Contribution of encoder, decoder, skip connections, residual blocks.

Part 1: U-Net architecture



The U-Net is a symmetric encoder–decoder convolutional network whose output has the same size as its input. In DDPM it maps $(x_t, t) \mapsto \boldsymbol{\epsilon}_\theta(x_t, t)$.

Major components and their functions:

1. **Encoder (contracting path):** Repeated [ResBlock(s) → downsample (stride-2 conv or pooling)]. Spatial size halves and channels increase at each level. It extracts increasingly abstract, larger-context features (edges → textures → object parts).
2. **Bottleneck:** Lowest resolution, highest channel count. Usually ResBlocks plus self-attention, giving a global view of the whole image so that the network understands overall structure and the global noise pattern.
3. **Decoder (expanding path):** Repeated [upsample (nearest/transposed conv) → concatenate skip → ResBlock(s)]. Restores spatial resolution back to the input size.
4. **Skip connections:** Concatenate encoder feature maps with decoder feature maps at the same resolution, passing fine spatial detail directly across.
5. **Residual blocks:** GroupNorm → SiLU → Conv → (add time embedding) → GroupNorm → SiLU → Conv, with output $y = x + F(x)$.
6. **Attention layers:** Self-attention at low resolutions (e.g. 16×16, 8×8) for long-range dependencies; cross-attention for text conditioning (Q10).
7. **Timestep embedding:** Sinusoidal encoding of t → MLP → added (or used as scale/shift) in every ResBlock, so one network handles all noise levels.
8. **Output layer:** GroupNorm → SiLU → 3×3 conv to 3 channels, giving the predicted noise with the same shape as the image.

Part 2: Numerical

Formula: Number of feature elements $= H \times W \times C$.

(a) Feature elements in Encoder Block 2 and Bottleneck

$$\text{Encoder Block 2} = 16 \times 16 \times 128 = 256 \times 128 = 32{,}768$$
$$\text{Bottleneck} = 4 \times 4 \times 512 = 16 \times 512 = 8{,}192$$

Encoder Block 2: 32,768 elements; Bottleneck: 8,192 elements (4 times fewer than Encoder Block 2).

(b) Dimensions after concatenation

Concatenation is along the channel axis; the spatial sizes must match (8×8 in both):

$$(8 \times 8 \times 256) \oplus (8 \times 8 \times 256) = 8 \times 8 \times (256 + 256) = 8 \times 8 \times 512$$

Concatenated feature map: $8 \times 8 \times 512$.

(c) Elements after vs before concatenation

$$\begin{aligned}\text{After concatenation} &= 8 \times 8 \times 512 = 64 \times 512 = 32{,}768\\ \text{Upsampled bottleneck (before)} &= 8 \times 8 \times 256 = 64 \times 256 = 16{,}384\\ \text{Encoder Block 3 (skip)} &= 8 \times 8 \times 256 = 16{,}384\end{aligned}$$
$$\text{Ratio} = \frac{32{,}768}{16{,}384} = 2$$

After concatenation: 32,768 elements, which is 2× the 16,384 elements of the upsampled feature map alone (= 16,384 + 16,384). Spatial size stays 8×8; only the channel count doubles. (Compared with the original bottleneck of 8,192 elements, it is 4× larger.)

A following convolution in Decoder Block 1 reduces the channels from 512 back to 256, giving the listed $8 \times 8 \times 256$ output.

(d) Output of the residual connection

$$\begin{aligned}y = x + F(x) &= [0.2, 0.4, 0.6] + [0.1, -0.2, 0.3]\\ &= [0.2 + 0.1,\ 0.4 - 0.2,\ 0.6 + 0.3] = [0.3,\ 0.2,\ 0.9]\end{aligned}$$

Residual output $y = [0.3,\ 0.2,\ 0.9]$

(e) MSE between actual and predicted noise

$$\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta = [0.2 - 0.1,\ 0.5 - 0.4,\ -0.3 - (-0.1),\ 0.1 - 0.2] = [0.1,\ 0.1,\ -0.2,\ -0.1]$$
$$\text{Squares} = [0.01,\ 0.01,\ 0.04,\ 0.01]; \quad \text{Sum} = 0.07$$
$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(\epsilon_i - \epsilon_{\theta,i})^2 = \frac{0.07}{4} = 0.0175$$

MSE = 0.0175 (sum of squared errors = 0.07)

(f) Contribution of each component to noise prediction

- **Encoder blocks:** Progressively downsample $64 \times 64 \to 32 \to 16 \to 8$ while increasing channels $3 \to 64 \to 128 \to 256$. They enlarge the receptive field and learn multi-scale features, separating the image content from the noise at different scales (fine noise at high resolution, structure at low resolution).

- **Bottleneck:** At $4 \times 4 \times 512$ it captures global context of the whole image; with self-attention it relates distant regions, which is crucial at high noise levels where only coarse structure remains.
- **Decoder blocks:** Upsample back to 64×64 and progressively turn the abstract features into a pixel-accurate noise map of the same size as the input, which is required because $\boldsymbol{\epsilon}_\theta$ must be predicted for every pixel.
- **Skip connections:** Downsampling loses fine spatial details; concatenating encoder features (e.g. $8 \times 8 \times 256 + 8 \times 8 \times 256 \to 8 \times 8 \times 512$) gives the decoder the exact high-resolution information it needs to localize the noise precisely. They also shorten gradient paths.
- **Residual blocks:** $y = x + F(x)$ (e.g. $[0.2, 0.4, 0.6] \to [0.3, 0.2, 0.9]$) lets each block learn only a correction to its input, allowing deep networks to train stably without vanishing gradients. They are also where the timestep embedding is injected, so the amount of denoising adapts to the noise level t.
- **Training signal:** All components are trained jointly by minimizing the MSE between true and predicted noise (e.g. 0.0175 in part e).

Q5. Denoising Score Matching (DSM)

Explain the concept and working principle of Denoising Score Matching. $x_t = 2.5$, $s_\theta(x_t, t) = -1.2$, $s^*(x_t, t) = -1.5$, $L = \|s_\theta - s^*\|_2^2$. (a) Define the score function and compute the prediction error. (b) DSM loss. (c) New loss if $s_\theta = -1.45$, and % reduction. (d) Actual and predicted noise before and after update using $\boldsymbol{\epsilon}_\theta = -\sigma s_\theta$, $\sigma = 0.5$. (e) How minimizing the loss helps learn the reverse process.

Part 1: Concept and working principle

Score function: For a probability density $p(x)$, the score is the gradient of the log-density with respect to the data:

$$s(x) = \nabla_x \log p(x).$$

It is a vector field pointing in the direction in which the probability density increases fastest, i.e. towards regions of realistic data. It does not need the normalizing constant Z: if $p(x) = \tilde{p}(x)/Z$, then $\nabla_x \log p(x) = \nabla_x \log \tilde{p}(x)$.

Explicit score matching would train $s_\theta(x) \approx \nabla_x \log p_{\text{data}}(x)$ by minimizing $\mathbb{E}_{p_{\text{data}}}[\|s_\theta(x) - \nabla_x \log p_{\text{data}}(x)\|^2]$, but the true score is unknown.

Denoising Score Matching (Vincent, 2011):

1. Corrupt clean data with known Gaussian noise: $\tilde{x} = x + \sigma\epsilon$, $\epsilon \sim \mathcal{N}(0, \mathbf{I})$, so $q_\sigma(\tilde{x} \mid x) = \mathcal{N}(\tilde{x}; x, \sigma^2\mathbf{I})$.
2. The score of this *known* conditional is easy to compute:

$$\nabla_{\tilde{x}} \log q_\sigma(\tilde{x} \mid x) = -\frac{\tilde{x} - x}{\sigma^2} = -\frac{\epsilon}{\sigma}.$$

3. Train the network on this target:

$$L_{\text{DSM}}(\theta) = \mathbb{E}_{x \sim p_{\text{data}},\, \epsilon}\left[\left\|s_\theta(\tilde{x}, \sigma) + \tfrac{\epsilon}{\sigma}\right\|^2\right].$$

4. **Key result:** this objective equals the explicit score-matching objective for the noisy marginal $q_\sigma(\tilde{x})$ up to a constant independent of θ. Hence its minimizer is $s_\theta(\tilde{x}) = \nabla_{\tilde{x}} \log q_\sigma(\tilde{x})$, the true score of the noisy data distribution – learned without ever knowing p_{data}.
5. **Multiple noise levels:** Using many σ (or timesteps t) and a network $s_\theta(x_t, t)$ gives scores at every noise level (NCSN, score-based SDE models).
6. **Link to DDPM:** with $\sigma_t = \sqrt{1 - \bar{\alpha}_t}$, $s_\theta(x_t, t) = -\boldsymbol{\epsilon}_\theta(x_t, t)/\sigma_t$; the DDPM noise-prediction loss is a weighted DSM loss.
7. **Sampling:** Langevin dynamics, $x \leftarrow x + \frac{\eta}{2} s_\theta(x) + \sqrt{\eta}\, z$, or the reverse SDE, move noise samples step by step towards high-density data regions.

Part 2: Numerical

Given: $x_t = 2.5$, $s_\theta(x_t, t) = -1.2$, $s^*(x_t, t) = -1.5$, $\sigma = 0.5$.

(a) Score function and prediction error

$$s(x_t, t) = \nabla_{x_t} \log p_t(x_t), \qquad s^*(x_t, t) = \nabla_{x_t} \log q(x_t \mid x_0) = -\frac{x_t - \sqrt{\bar{\alpha}_t}\, x_0}{1 - \bar{\alpha}_t} = -\frac{\epsilon}{\sigma_t}.$$

$$\text{Error} = s_\theta(x_t, t) - s^*(x_t, t) = -1.2 - (-1.5) = -1.2 + 1.5 = 0.3$$

Score prediction error = 0.3 (absolute error 0.3, i.e. 0.3/1.5 = 20% relative error). The predicted score −1.2 is less negative than the target, i.e. it underestimates the pull of $x_t = 2.5$ back towards higher-density (smaller) values.

(b) DSM loss

$$L = \|s_\theta - s^*\|_2^2 = (0.3)^2 = 0.09$$

$L = 0.09$

(c) Loss after the update $s_\theta = -1.45$

$$\text{New error} = -1.45 - (-1.5) = 0.05$$
$$L' = (0.05)^2 = 0.0025$$
$$\text{Reduction} = \frac{L - L'}{L} \times 100\% = \frac{0.09 - 0.0025}{0.09} \times 100\% = \frac{0.0875}{0.09} \times 100\% = 97.22\%$$

New loss $L' = 0.0025$; percentage reduction $\approx 97.22\%$ (36× smaller).

(d) Noise values using $\epsilon = -\sigma s$, $\sigma = 0.5$

$$\text{Actual noise: } \epsilon = -\sigma\, s^* = -0.5 \times (-1.5) = 0.75$$
$$\text{Predicted (before): } \epsilon_\theta = -\sigma\, s_\theta = -0.5 \times (-1.2) = 0.60$$
$$\text{Predicted (after): } \epsilon'_\theta = -\sigma\, s'_\theta = -0.5 \times (-1.45) = 0.725$$

	Score	Noise $= -\sigma \times$score	Noise error	Squared noise error
Target / actual	−1.5	0.75	–	–
Before update	−1.2	0.60	$0.75 - 0.60 = 0.15$	0.0225
After update	−1.45	0.725	$0.75 - 0.725 = 0.025$	0.000625

Actual noise = 0.75. Predicted noise: 0.60 before, 0.725 after the update. The noise error falls from 0.15 to 0.025; squared noise error = $\sigma^2\times$ score loss: $0.25 \times 0.09 = 0.0225$ and $0.25 \times 0.0025 = 0.000625$ (same 97.22% reduction).

(e) How minimizing the score-matching loss teaches the reverse process

1. **The reverse process needs the score:** The reverse-time SDE (Anderson, 1982) is $dx = [f(x,t) - g(t)^2 \nabla_x \log p_t(x)]\,dt + g(t)\,d\bar{w}$. The only unknown quantity is the score $\nabla_x \log p_t(x)$. Minimizing the DSM loss makes $s_\theta(x_t, t) \approx \nabla_x \log p_t(x_t)$ at every noise level, so the reverse process can be simulated.
2. **Direction of denoising:** The score points from a noisy sample towards higher data density, i.e. towards cleaner data. Here $s^* = -1.5 < 0$ tells the model to move $x_t = 2.5$ in the negative direction. A more accurate score (−1.45 vs −1.2) gives a more correct step size and direction.
3. **Equivalence with noise prediction:** $\boldsymbol{\epsilon}_\theta = -\sigma s_\theta$, so a better score gives a better noise estimate (0.725 vs 0.60, true 0.75), and therefore a better DDPM reverse mean $\mu_\theta = \frac{1}{\sqrt{\alpha_t}}(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\boldsymbol{\epsilon}_\theta)$ and a better clean-image estimate $\hat{x}_0$ (Tweedie's formula: $\mathbb{E}[x_0 \mid x_t] = (x_t + (1-\bar{\alpha}_t)\nabla \log p_t(x_t))/\sqrt{\bar{\alpha}_t}$).
4. **Multi-scale learning:** Training over all noise levels gives accurate scores both far from the data (large noise, global structure) and close to it (small noise, fine detail), so sampling can start from pure noise and end at realistic samples.
5. **Error accumulation:** Small score errors at each of many steps add up; reducing the loss (here by 97%) reduces drift and improves sample quality.

Q6. Denoising Diffusion Implicit Models (DDIM)

Explain DDIM and its non-Markovian formulation, the significance of σ_t, and deterministic sampling when $\sigma_t = 0$. DDPM: 1000 steps; DDIM: 50 steps; 25 ms per U-Net evaluation; other costs negligible. (a) DDPM time. (b) DDIM time. (c) % reduction. (d) Time saved for 200 images.

Part 1: DDIM and the non-Markovian formulation

Motivation: DDPM sampling needs $T = 1000$ sequential network evaluations because its reverse process inverts a Markov chain step by step. DDIM (Song, Meng & Ermon, 2021) produces high-quality samples in 10–100 steps *using the same trained DDPM network*, with no retraining.

Key observation: The DDPM training loss depends only on the marginals

$$q(x_t \mid x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}\, x_0,\ (1-\bar{\alpha}_t)\mathbf{I}),$$

not on the joint Markov chain. Therefore any forward process with the same marginals has the same training objective.

Non-Markovian forward process: DDIM defines a family of forward processes indexed by the vector $\sigma = (\sigma_1, \ldots, \sigma_T)$ of standard deviations:

$$q_\sigma(x_{t-1} \mid x_t, x_0) = \mathcal{N}(x_{t-1};\ \mu_\sigma,\ \sigma_t^2\mathbf{I}),$$

where

$$\mu_\sigma = \sqrt{\bar{\alpha}_{t-1}}\, x_0 + \sqrt{1-\bar{\alpha}_{t-1}-\sigma_t^2}\cdot\frac{x_t - \sqrt{\bar{\alpha}_t}\, x_0}{\sqrt{1-\bar{\alpha}_t}}.$$

Here each x_t may depend on both x_{t-1} and x_0, so the forward process is *non-Markovian*, but the marginals $q_\sigma(x_t \mid x_0)$ are still $\mathcal{N}(\sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_t)\mathbf{I})$.

DDIM sampling step: Using the network's prediction $\boldsymbol{\epsilon}_\theta(x_t, t)$:

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}}\underbrace{\left(\frac{x_t - \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}_\theta(x_t,t)}{\sqrt{\bar{\alpha}_t}}\right)}_{\text{predicted } \hat{x}_0} + \underbrace{\sqrt{1-\bar{\alpha}_{t-1}-\sigma_t^2}\,\boldsymbol{\epsilon}_\theta(x_t,t)}_{\text{direction pointing to } x_t} + \underbrace{\sigma_t z}_{\text{random noise}}.$$

Skipping steps: Because the update is defined through $\hat{x}_0$ and the marginals, it can be applied on a sub-sequence $\tau = [\tau_1, \ldots, \tau_S]$ of timesteps (e.g. $S = 50$ out of 1000: $t = 1000, 980, 960, \ldots$), replacing $t-1$ by the previous element of τ.

Significance of σ_t: It controls the amount of randomness. It is usually written as

$$\sigma_t(\eta) = \eta\sqrt{\frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}}\sqrt{1-\frac{\bar{\alpha}_t}{\bar{\alpha}_{t-1}}}.$$

- $\eta = 1$: the process becomes the original Markovian DDPM (stochastic).
- $0 < \eta < 1$: intermediate stochasticity.
- $\eta = 0 \Rightarrow \sigma_t = 0$: **deterministic DDIM**.

Deterministic sampling when $\sigma_t = 0$:

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}}\,\hat{x}_0(x_t) + \sqrt{1-\bar{\alpha}_{t-1}}\,\boldsymbol{\epsilon}_\theta(x_t,t).$$

- No random noise is added; the same x_T always gives the same x_0. The model becomes an *implicit* generative model (a deterministic mapping from noise to data), hence the name.

- It corresponds to solving the probability-flow ODE with a first-order (Euler-like) method, which allows large steps with little error, so 20–50 steps achieve quality close to 1000-step DDPM.
- Consistency: the latent x_T acts as an encoding of the image, enabling inversion (image → noise → image), image editing and meaningful interpolation in latent space.

Part 2: Numerical

Given: $T_{\text{DDPM}} = 1000$ steps, $T_{\text{DDIM}} = 50$ steps, 25 ms per U-Net evaluation (one evaluation per step).
Formula: Sampling time = number of steps × time per evaluation.

(a) DDPM sampling time

$$t_{\text{DDPM}} = 1000 \times 25 \text{ ms} = 25{,}000 \text{ ms} = 25 \text{ s}$$

DDPM: 25,000 ms = 25 s per image.

(b) DDIM sampling time

$$t_{\text{DDIM}} = 50 \times 25 \text{ ms} = 1{,}250 \text{ ms} = 1.25 \text{ s}$$

DDIM: 1,250 ms = 1.25 s per image.

(c) Percentage reduction

$$\text{Reduction} = \frac{25{,}000 - 1{,}250}{25{,}000} \times 100\% = \frac{23{,}750}{25{,}000} \times 100\% = 95\%$$

95% reduction in sampling time (DDIM is 25,000/1,250 = 20× faster).

(d) Time saved for 200 images

$$\begin{aligned} \text{DDPM total} &= 200 \times 25 \text{ s} = 5{,}000 \text{ s} \\ \text{DDIM total} &= 200 \times 1.25 \text{ s} = 250 \text{ s} \\ \text{Time saved} &= 5{,}000 - 250 = 4{,}750 \text{ s} \end{aligned}$$

Converting: $4{,}750 \text{ s} = \frac{4{,}750}{60} = 79.17 \text{ min} = 79 \text{ min } 10 \text{ s} \approx 1.32 \text{ hours}$.

Time saved = 4,750 s (= 4,750,000 ms) ≈ 79.2 minutes ≈ 1.32 hours.

Q7. Classifier Guidance and Classifier-Free Guidance

Explain classifier guidance and classifier-free guidance; how classifier information enters the reverse process. $s_{\text{guided}} = s_{\text{uncond}} + \omega(s_{\text{cond}} - s_{\text{uncond}})$ with $s_{\text{uncond}} = [0.2, 0.4, 0.6]$, $s_{\text{cond}} = [0.8, 0.6, 0.2]$. (a)–(c) Guided score for $\omega = 1, 2, 3$. (d) Euclidean distance between guided and unconditional score for each ω. (e) Effect of ω on quality and diversity.

Part 1: Conditional generation via the score

To generate a sample of class/condition y, we need the conditional score $\nabla_{x_t} \log p(x_t \mid y)$. By Bayes' rule $p(x_t \mid y) = \dfrac{p(y \mid x_t)\, p(x_t)}{p(y)}$, and since $p(y)$ does not depend on x_t:

$$\nabla_{x_t} \log p(x_t \mid y) = \underbrace{\nabla_{x_t} \log p(x_t)}_{\text{unconditional score}} + \underbrace{\nabla_{x_t} \log p(y \mid x_t)}_{\text{classifier gradient}}.$$

Part 2: Classifier guidance (Dhariwal & Nichol, 2021)

1. Train a separate classifier $p_\phi(y \mid x_t, t)$ on *noisy* images at all timesteps (an ordinary classifier fails on noisy inputs).
2. At each reverse step, compute the gradient of the log-probability of the desired class with respect to the noisy image, $\nabla_{x_t} \log p_\phi(y \mid x_t)$, and add it with a guidance scale s:

$$\tilde{s}(x_t, y) = \nabla_{x_t} \log p(x_t) + s\, \nabla_{x_t} \log p_\phi(y \mid x_t).$$

3. **In the reverse process** (mean shift form): $x_{t-1} \sim \mathcal{N}(\mu_\theta + s\, \Sigma_t \nabla_{x_t} \log p_\phi(y \mid x_t),\ \Sigma_t)$; in noise form: $\hat{\epsilon} = \boldsymbol{\epsilon}_\theta(x_t) - s\sqrt{1 - \bar{\alpha}_t}\, \nabla_{x_t} \log p_\phi(y \mid x_t)$.
4. The classifier gradient pushes each denoising step towards images that the classifier recognizes as class y. A large s corresponds to sampling from $p(x_t)\, p(y \mid x_t)^s$, a sharper distribution.
5. **Drawbacks:** needs an extra classifier trained on noisy data; gradients can be adversarial (fooling the classifier rather than improving the image); only works for labels a classifier can model.

Part 3: Classifier-free guidance (Ho & Salimans, 2022)

1. Train *one* network $\boldsymbol{\epsilon}_\theta(x_t, t, c)$ for both conditional and unconditional generation: during training the condition c is replaced by a null token $\varnothing$ with some probability (e.g. 10–20%, "condition dropout").
2. At sampling, evaluate both $\boldsymbol{\epsilon}_\theta(x_t, t, c)$ and $\boldsymbol{\epsilon}_\theta(x_t, t, \varnothing)$ and combine:

$$\hat{\epsilon} = \boldsymbol{\epsilon}_\theta(x_t, \varnothing) + \omega\big(\boldsymbol{\epsilon}_\theta(x_t, c) - \boldsymbol{\epsilon}_\theta(x_t, \varnothing)\big) \quad \Longleftrightarrow \quad s_{\text{guided}} = s_{\text{uncond}} + \omega\,(s_{\text{cond}} - s_{\text{uncond}}).$$

3. The difference $s_{\text{cond}} - s_{\text{uncond}}$ acts as an *implicit classifier* gradient: since $\nabla \log p(c \mid x_t) = \nabla \log p(x_t \mid c) - \nabla \log p(x_t)$, no separate classifier is needed.
4. $\omega = 0$: unconditional; $\omega = 1$: plain conditional; $\omega > 1$: extrapolates beyond the conditional score, strengthening the condition (Stable Diffusion uses $\omega \approx 7.5$).
5. **Advantages:** simpler (one model), works with any conditioning (text, images), gives better quality; cost: two forward passes per step (often batched).

Part 4: Numerical

Given: $s_u = [0.2, 0.4, 0.6]$, $s_c = [0.8, 0.6, 0.2]$.
Guidance direction:

$$\Delta = s_c - s_u = [0.8 - 0.2,\ 0.6 - 0.4,\ 0.2 - 0.6] = [0.6,\ 0.2,\ -0.4]$$

So $s_{\text{guided}} = s_u + \omega\Delta$.

(a) $\omega = 1$

$$s_{\text{guided}} = [0.2, 0.4, 0.6] + 1 \times [0.6, 0.2, -0.4] = [0.8,\ 0.6,\ 0.2]$$

$\omega = 1$: $s_{\text{guided}} = [0.8,\ 0.6,\ 0.2]$ (exactly the conditional score).

(b) $\omega = 2$

$$s_{\text{guided}} = [0.2, 0.4, 0.6] + 2 \times [0.6, 0.2, -0.4] = [0.2 + 1.2,\ 0.4 + 0.4,\ 0.6 - 0.8] = [1.4,\ 0.8,\ -0.2]$$

$\omega = 2$: $s_{\text{guided}} = [1.4,\ 0.8,\ -0.2]$

(c) $\omega = 3$

$$s_{\text{guided}} = [0.2, 0.4, 0.6] + 3 \times [0.6, 0.2, -0.4] = [0.2 + 1.8,\ 0.4 + 0.6,\ 0.6 - 1.2] = [2.0,\ 1.0,\ -0.6]$$

$\omega = 3$: $s_{\text{guided}} = [2.0,\ 1.0,\ -0.6]$

(d) Euclidean distance from the unconditional score

Since $s_{\text{guided}} - s_u = \omega\Delta$, the distance is $d = \|\omega\Delta\|_2 = \omega\|\Delta\|_2$, where

$$\|\Delta\|_2 = \sqrt{0.6^2 + 0.2^2 + (-0.4)^2} = \sqrt{0.36 + 0.04 + 0.16} = \sqrt{0.56} = 0.7483.$$

Direct computation for each scale:

$$\omega = 1 : s_g - s_u = [0.6, 0.2, -0.4], \quad d = \sqrt{0.36 + 0.04 + 0.16} = \sqrt{0.56} = 0.7483$$
$$\omega = 2 : s_g - s_u = [1.2, 0.4, -0.8], \quad d = \sqrt{1.44 + 0.16 + 0.64} = \sqrt{2.24} = 1.4967$$
$$\omega = 3 : s_g - s_u = [1.8, 0.6, -1.2], \quad d = \sqrt{3.24 + 0.36 + 1.44} = \sqrt{5.04} = 2.2450$$

ω	Guided score	$s_g - s_u$	Distance $\|s_g - s_u\|_2$
1	$[0.8, 0.6, 0.2]$	$[0.6, 0.2, -0.4]$	0.7483
2	$[1.4, 0.8, -0.2]$	$[1.2, 0.4, -0.8]$	1.4967
3	$[2.0, 1.0, -0.6]$	$[1.8, 0.6, -1.2]$	2.2450

Distances: 0.7483, 1.4967, 2.2450 for $\omega = 1, 2, 3$. The distance grows *linearly* with ω ($= \omega \times 0.7483$): the guided score moves further from the unconditional score along the conditional direction.

(e) Effect of increasing the guidance scale

- **Stronger conditioning:** Larger ω pushes the sampling trajectory more strongly towards the conditional direction (as seen by the linearly increasing distance). Samples match the class or text prompt more closely (higher CLIP score / prompt adherence).
- **Higher quality (fidelity):** Samples become sharper, more prototypical and more recognizable; Inception Score and precision increase. Effectively, sampling is from $p(x)\, p(c \mid x)^{\omega}$, concentrating on high-likelihood, typical images.

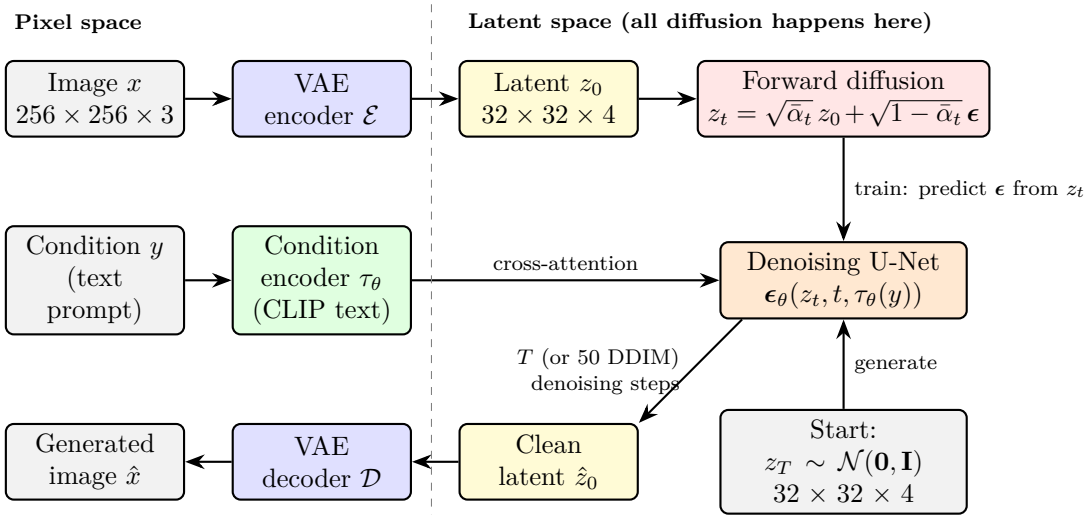
- **Lower diversity:** The distribution becomes sharper (mode-seeking), so different seeds produce more similar images; recall decreases.
- **Trade-off:** FID first improves (moderate ω, e.g. 1.5–3 for class conditioning, 5–10 for text) and then worsens. Very large ω causes over-saturated colours, exaggerated contrast, artefacts and unnatural images, because the extrapolated score leaves the region of realistic data.
- $\omega = 0$: unconditional (maximum diversity, ignores the condition); $\omega = 1$: standard conditional sampling; $\omega > 1$: quality gained at the cost of diversity.

Q8. Latent Diffusion Models (LDM)

Explain the architecture and working of latent diffusion models; how a VAE is combined with diffusion in latent space; advantages over pixel-space diffusion. VAE compresses $256 \times 256 \times 3$ to $32 \times 32 \times 4$. (a) Elements in the image. (b) Elements in the latent. (c) Compression ratio. (d) % reduction in elements processed. (e) How latent diffusion reduces computation.

Part 1: Architecture and working principle

Latent Diffusion Models (Rombach et al., 2022; the basis of Stable Diffusion) separate image generation into two stages: *perceptual compression* by an autoencoder and *semantic generation* by a diffusion model in the compressed latent space.



(Top and middle rows: training path. Bottom row: generation path, starting from noise at the bottom right.)

Stage 1 – Autoencoder (VAE) training (done once):

- Encoder $\mathcal{E}$ maps image $x \in \mathbb{R}^{H \times W \times 3}$ to latent $z = \mathcal{E}(x) \in \mathbb{R}^{h \times w \times c}$ with downsampling factor $f = H/h$ (e.g. $f = 8$).
- Decoder $\mathcal{D}$ reconstructs $\tilde{x} = \mathcal{D}(z)$.
- Trained with reconstruction loss + perceptual (LPIPS) loss + patch-based adversarial loss + a small KL regularization towards $\mathcal{N}(0, \mathbf{I})$ (or vector quantization). This removes imperceptible high-frequency detail while keeping perceptually important content, and keeps the latent well-scaled for diffusion.

Stage 2 – Diffusion in latent space:

- The VAE is frozen. Every training image is encoded, $z_0 = \mathcal{E}(x)$, and a standard DDPM is trained on z_0:

$$L_{\text{LDM}} = \mathbb{E}_{\mathcal{E}(x),\, y,\, \epsilon,\, t}\Big[\big\|\epsilon - \epsilon_\theta(z_t, t, \tau_\theta(y))\big\|_2^2\Big].$$

- The U-Net includes cross-attention layers to condition on text, layouts, etc. via an encoder τ_θ.

Generation: Sample $z_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ in latent shape; denoise with the U-Net for T (or fewer, with DDIM) steps to get $\hat{z}_0$; decode once: $\hat{x} = \mathcal{D}(\hat{z}_0)$.

Advantages over pixel-space diffusion:

1. **Much lower computation and memory** for training and sampling (fewer elements per U-Net pass; see numerical part), enabling training on fewer GPUs and running on consumer hardware.
2. **High-resolution synthesis** (512×512 and above) becomes practical.
3. **Focus on semantics:** The VAE handles fine pixel detail; the diffusion model spends its capacity on the meaningful content and composition rather than imperceptible details.
4. **Reusable autoencoder:** One VAE serves many diffusion models and tasks.
5. **Flexible conditioning** via cross-attention: text-to-image, inpainting, super-resolution, layout-to-image.
6. Comparable or better image quality (FID) than pixel-space models at a fraction of the cost.

Part 2: Numerical

Given: Image $256 \times 256 \times 3$; latent $32 \times 32 \times 4$.

(a) Elements in the original image

$$N_x = 256 \times 256 \times 3 = 65{,}536 \times 3 = 196{,}608$$

196,608 elements

(b) Elements in the latent

$$N_z = 32 \times 32 \times 4 = 1{,}024 \times 4 = 4{,}096$$

4,096 elements

(c) Compression ratio

$$\text{Compression ratio} = \frac{N_x}{N_z} = \frac{196{,}608}{4{,}096} = 48$$

(Spatial downsampling factor $f = 256/32 = 8$ in each dimension, i.e. $8^2 = 64\times$ fewer spatial positions, with channels changing from 3 to 4: $64 \times \frac{3}{4} = 48$.)

Compression ratio $= 48 : 1$

(d) Percentage reduction in elements processed

$$\text{Reduction} = \frac{196{,}608 - 4{,}096}{196{,}608} \times 100\% = \frac{192{,}512}{196{,}608} \times 100\% = \left(1 - \frac{1}{48}\right) \times 100\% = 97.92\%$$

97.92% fewer elements (the diffusion model processes only $\approx$ 2.08% of the original element count).

(e) How latent-space diffusion reduces computation

1. **Smaller input per step:** Each U-Net evaluation processes 4,096 values instead of 196,608 (48× fewer). Convolution cost scales with the number of spatial positions, so a 32×32 input costs about 64× fewer spatial operations per layer than 256×256.
2. **Cheaper attention:** Self-attention cost grows as $O(n^2)$ in the number of positions n. With $n = 32 \times 32 = 1{,}024$ instead of $256 \times 256 = 65{,}536$, the attention matrix is $64^2 = 4{,}096$ times smaller – full-resolution attention on pixels would be infeasible.

3. **Cost multiplied by the number of steps:** Diffusion needs tens to thousands of U-Net evaluations per image; the savings apply to every step and every training iteration, while the expensive decoder runs only *once* at the end (and the encoder once per training image).
4. **Lower memory:** Smaller activations allow larger batch sizes, faster training and inference on a single consumer GPU.
5. **No waste on imperceptible detail:** The VAE already handles high-frequency detail, so the diffusion model needs fewer parameters and steps to achieve good perceptual quality.

Q9. Accelerating Diffusion Model Sampling

Discuss approaches for accelerating diffusion sampling: DDIM, progressive distillation, latent-space diffusion. Compare their principles and explain how they address the computational challenges of conventional diffusion models.

The computational challenge

Conventional DDPM sampling is slow because:

1. **Many sequential steps:** $T = 1000$ reverse steps, each requiring a full U-Net evaluation; steps cannot be parallelized because x_{t-1} depends on x_t.
2. **Expensive steps:** Each U-Net pass on high-resolution pixels (e.g. $256 \times 256 \times 3$) with convolutions and attention is costly; attention scales quadratically with resolution.

$$\text{Total sampling cost} = (\text{number of steps}) \times (\text{cost per step}).$$

DDIM and progressive distillation reduce the *number of steps*; latent diffusion reduces the *cost per step*.

1. Denoising Diffusion Implicit Models (DDIM)

- **Principle:** The DDPM objective depends only on the marginals $q(x_t \mid x_0)$, so a non-Markovian forward process with the same marginals can be used. Its reverse update is $x_{\tau_{i-1}} = \sqrt{\bar{\alpha}_{\tau_{i-1}}}\,\hat{x}_0 + \sqrt{1 - \bar{\alpha}_{\tau_{i-1}} - \sigma^2}\,\boldsymbol{\epsilon}_\theta + \sigma z$, where $\hat{x}_0 = (x_{\tau_i} - \sqrt{1 - \bar{\alpha}_{\tau_i}}\,\boldsymbol{\epsilon}_\theta)/\sqrt{\bar{\alpha}_{\tau_i}}$.
- **Acceleration:** The update can jump directly between non-adjacent timesteps of a sub-sequence τ (e.g. 50 of 1000). With $\sigma = 0$ it is deterministic and corresponds to an ODE solver, so large steps are accurate.
- **Speed-up:** 10× to 50× (e.g. 1000 → 50 steps gives 95% less time, Q6), with no retraining; the same pre-trained DDPM network is used.
- **Extras:** deterministic encoding/inversion, consistent outputs, semantic interpolation.
- **Limitations:** quality degrades for very few steps (< 10); still needs tens of evaluations.

2. Progressive distillation (Salimans & Ho, 2022)

- **Principle:** Knowledge distillation applied repeatedly. A *teacher* sampler (e.g. DDIM with N steps) is distilled into a *student* that performs *one* step equal to *two* teacher steps.
- **Procedure:**
  1. Initialize the student as a copy of the teacher.
  2. For a noisy input z_t, run two deterministic DDIM teacher steps $t \to t - 1/N \to t - 2/N$ to get the target $\tilde{z}$.
  3. Train the student to reach $\tilde{z}$ in a single step (usually by matching the implied $\tilde{x}_0$ target, with v-prediction for stability).
  4. The student, now using $N/2$ steps, becomes the new teacher; repeat.
- **Speed-up:** Halving at every round: $8192 \to 4096 \to \cdots \to 8 \to 4$ steps (even 1–2 steps), i.e. hundreds to thousands of times fewer evaluations, with FID close to the original model at 4–8 steps.
- **Limitations:** requires extra training rounds (computationally expensive) and a fixed sampler; quality drops at 1–2 steps; errors may compound across rounds.

3. Latent-space diffusion (Rombach et al., 2022)

- **Principle:** Perform diffusion in the compressed latent space of a pre-trained VAE instead of pixel space: $z = \mathcal{E}(x)$, diffuse/denoise z, decode $\hat{x} = \mathcal{D}(\hat{z}_0)$ once.
- **Acceleration:** Each step processes far fewer elements, e.g. $256\times256\times3 = 196{,}608 \to 32\times32\times4 = 4{,}096$ (48× fewer, Q8), and attention matrices become thousands of times smaller.
- **Speed-up:** reduces both training and sampling cost per step, enabling high-resolution generation on consumer GPUs (Stable Diffusion).
- **Limitations:** quality is upper-bounded by the VAE reconstruction (fine details such as small text or faces may be lost); needs a separately trained autoencoder; the number of steps is unchanged unless combined with DDIM/distillation.

Comparison

Aspect	DDIM	Progressive distillation	Latent diffusion
What is reduced	Number of steps	Number of steps	Cost per step
Core idea	Non-Markovian process with same marginals; deterministic ODE-like sampler on a sub-sequence of timesteps	Student learns to do two teacher steps in one; repeated halving	Diffuse in a compressed VAE latent space
Retraining needed	No (uses pre-trained DDPM)	Yes (several distillation rounds)	Yes (train VAE + diffusion in latent space)
Typical steps	20–100	4–8 (even 1–2)	Same as base sampler (e.g. 50 with DDIM)
Typical speed-up	10–50×	100–1000× vs. original	$\sim$ 48–64× fewer elements per step
Stochasticity	Deterministic ($\sigma = 0$) or tunable via η	Deterministic	Any sampler
Main limitation	Quality drops below $\sim$10 steps	Training cost; slight quality loss	Limited by VAE fidelity

Combined use: These methods are complementary. Stable Diffusion combines latent diffusion (cheap steps) with DDIM-type samplers (few steps); distilled latent models (e.g. few-step distilled Stable Diffusion) combine all three ideas, reducing total generation time from minutes to well under a second.

Q10. Conditional Diffusion Models and Text Conditioning

Explain conditional diffusion models and how different types of conditioning information are incorporated into the U-Net. Discuss an application with the mechanisms used for text conditioning and cross-attention.

Part 1: Conditional diffusion models

An unconditional diffusion model learns $p(x)$ and generates random samples. A **conditional diffusion model** learns $p(x \mid c)$, where c is extra information that controls what is generated (class label, text, image, mask, pose, depth map, audio, etc.). The noise-prediction network receives the condition as an additional input:

$$\boldsymbol{\epsilon}_\theta(x_t, t, c), \qquad L = \mathbb{E}_{x_0,c,\boldsymbol{\epsilon},t}\Big[\big\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \boldsymbol{\epsilon},\ t,\ c)\big\|^2\Big].$$

The reverse process becomes $p_\theta(x_{t-1} \mid x_t, c)$, so every denoising step is steered towards samples consistent with c. Guidance (classifier or classifier-free, Q7) can strengthen the effect of c at sampling time.

Part 2: Incorporating conditioning into the U-Net

1. **Timestep conditioning (always present):** $t \to$ sinusoidal embedding $\to$ MLP $\to$ vector e_t, which is added to the features inside every residual block, or used for adaptive normalization:

$$h' = \gamma(e) \cdot \mathrm{GroupNorm}(h) + \beta(e) \qquad \text{(AdaGN / FiLM: scale and shift).}$$

2. **Class-label conditioning:** A learned embedding of the label y is added to the timestep embedding: $e = e_t + \mathrm{Embed}(y)$, and injected the same way (scale/shift in every ResBlock). Used for class-conditional ImageNet generation.

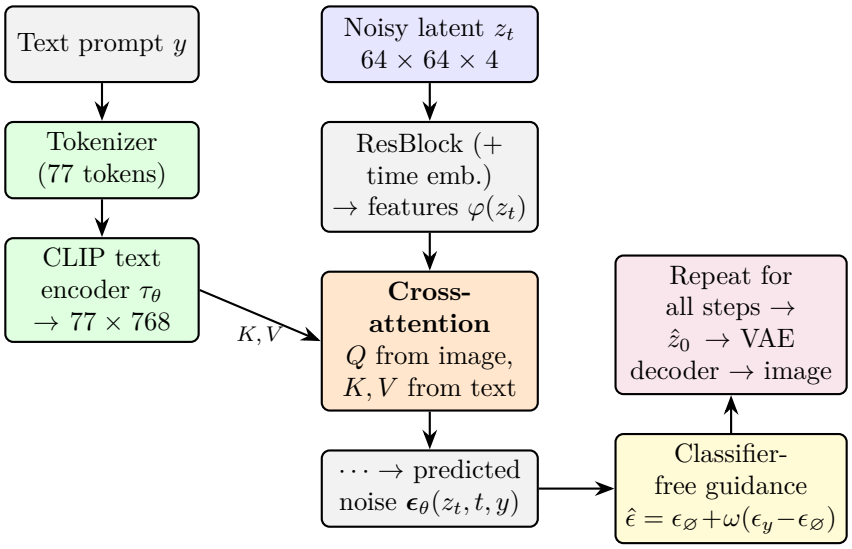
3. **Spatial / image conditioning (concatenation):** When the condition is an image of the same spatial layout (low-resolution image for super-resolution, masked image + mask for inpainting, depth/segmentation map, sketch), it is resized and *concatenated along the channel dimension* with x_t at the U-Net input: input $= [x_t;\ c]$ with $C_x + C_c$ channels. ControlNet uses a trainable copy of the encoder that takes the spatial condition and adds its features to the U-Net through zero-initialized convolutions.

4. **Sequence conditioning (cross-attention):** When the condition is a sequence (text tokens, image patch embeddings, audio), it is encoded by τ_θ and injected via *cross-attention* layers inside the U-Net blocks, where U-Net spatial features query the condition tokens. This is the most flexible mechanism and is used for text.

5. **Global embeddings:** A pooled text or image embedding (e.g. CLIP image embedding) can also be added to the timestep embedding, or appended as extra tokens to the cross-attention context.

Part 3: Application – Text-to-image generation (e.g. Stable Diffusion)

Task: Generate an image from a prompt such as "*a red bicycle leaning against a brick wall, oil painting*".



Text conditioning mechanism:

1. The prompt is tokenized (BPE) and padded/truncated to a fixed length (77 tokens in Stable Diffusion v1).
2. A frozen pre-trained text encoder τ_θ (CLIP ViT-L/14 text transformer; Imagen uses T5-XXL) converts it to a sequence of contextual embeddings $\tau_\theta(y) \in \mathbb{R}^{M \times d_\tau}$ (e.g. 77×768). Because CLIP was trained to align text with images, these embeddings carry visual semantics.

Cross-attention mechanism: Inside the U-Net, after ResBlocks and self-attention, each spatial transformer block contains a cross-attention layer. Let $\varphi_i(z_t) \in \mathbb{R}^{N \times d_\epsilon}$ be the flattened U-Net feature map ($N = h \times w$ spatial positions):

$$Q = W_Q^{(i)} \varphi_i(z_t), \qquad K = W_K^{(i)} \tau_\theta(y), \qquad V = W_V^{(i)} \tau_\theta(y),$$

$$\text{CrossAttention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V, \qquad QK^\top \in \mathbb{R}^{N \times M}.$$

- **Queries from the image** (each spatial position asks "which words are relevant to me?"); **Keys and Values from the text** (each word offers its meaning).
- The $N \times M$ attention map links pixels/regions to words: e.g. positions where the bicycle should appear attend strongly to "red" and "bicycle", background positions to "brick wall". The attended values are added back to the image features (residual), injecting the text meaning into the spatial layout.
- Cross-attention is applied at several resolutions, so text controls both global composition (low resolution) and details (high resolution).
- Text-conditioned training with condition dropout enables classifier-free guidance at sampling time ($\omega \approx 7.5$), which strengthens adherence to the prompt.

Generation pipeline:

1. Encode the prompt once: $\tau_\theta(y)$ (and the empty prompt for CFG).
2. Sample $z_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ of shape $64 \times 64 \times 4$ (for a 512×512 image).
3. For each of e.g. 50 DDIM steps: predict $\epsilon_\theta(z_t, t, y)$ and $\epsilon_\theta(z_t, t, \varnothing)$, combine with CFG, update $z_t \rightarrow z_{t-1}$.

4. Decode $\hat{x} = \mathcal{D}(\hat{z}_0)$ to obtain the 512×512 image.

Other applications: image editing and inpainting with text (InstructPix2Pix), super-resolution, text-to-video, text-to-audio, layout- or pose-controlled generation (ControlNet), medical image synthesis conditioned on labels.

End of solutions